

Assignment: Set Up FastAPI Project and Build Basic Ticket APIs

Project: AI-Powered Client Issue Triage Platform

Background

In this project, we are building a backend platform that helps a company manage user or client issues.

A user may report problems such as:

- "TSLA price has not updated for 30 minutes."
- "My watchlist disappeared."
- "The login page is not working."
- "Why does the stock price change after market close?"

In the full project, the system will eventually:

1. Create a ticket.
2. Select the correct workflow.
3. Create a workflow run.
4. Send the workflow run ID to SQS.
5. Use a worker to process the workflow asynchronously.

In this assignment, we will only implement the first part:

Set up the backend project and create basic Ticket APIs using FastAPI.

We will not connect to a real database yet. For now, we will store tickets in memory using a Python dictionary.

Learning Goals

By the end of this assignment, you should understand:

1. How to create a GitHub repository.
 2. How to set up a Python FastAPI project.
 3. How to structure a backend project using router, service, repository, and model layers.
 4. How to implement RESTful APIs.
 5. How to create and retrieve tickets.
 6. How to test APIs using FastAPI Swagger UI.
-

Part 1: Create GitHub Repository

Step 1: Create a new repository on GitHub

Go to GitHub and create a new repository.

Repository name:

```
ai-ticket-workflow-platform
```

Recommended settings:

```
Visibility: Public or Private  
Initialize with README: Yes  
Add .gitignore: Python  
License: No license needed for now
```

Step 2: Clone the repository locally

Open your terminal and run:

```
git clone https://github.com/YOUR_USERNAME/ai-ticket-workflow-platform.git
```

Then go into the project folder:

```
cd ai-ticket-workflow-platform
```

Part 2: Set Up Python Environment

Step 3: Create a virtual environment

Run:

```
python3 -m venv venv
```

Activate it:

Mac / Linux

```
source venv/bin/activate
```

Windows

```
venv\Scripts\activate
```

After activation, your terminal should show something like:

```
(venv)
```

Step 4: Create requirements.txt

Create a file named:

```
requirements.txt
```

Add:

```
fastapi  
uvicorn  
pydantic
```

Install dependencies:

```
pip install -r requirements.txt
```

Part 3: Create Project Structure

Step 5: Create folders and files

Create this project structure:

```
ai-ticket-workflow-platform/  
  app/  
    __init__.py  
    main.py  
  
    models/  
      __init__.py  
      ticket_models.py  
  
    routers/  
      __init__.py  
      ticket_router.py  
  
    services/  
      __init__.py  
      ticket_service.py  
  
    repositories/  
      __init__.py  
      ticket_repository.py  
  
  requirements.txt  
  README.md
```

You can create the folders using:

```
mkdir -p app/models app/routers app/services app/repositories
```

Create the Python files:

```
touch app/__init__.py  
touch app/main.py  
touch app/models/__init__.py  
touch app/models/ticket_models.py  
touch app/routers/__init__.py  
touch app/routers/ticket_router.py  
touch app/services/__init__.py  
touch app/services/ticket_service.py  
touch app/repositories/__init__.py  
touch app/repositories/ticket_repository.py
```

Part 4: Understand the Project Layers

Before writing code, understand what each layer does.

1. Router Layer

The router layer handles HTTP requests.

Example:

```
POST /tickets
GET /tickets/{ticket_id}
```

The router should not contain complex business logic.

2. Service Layer

The service layer contains business logic.

For example:

```
Create a ticket ID.
Set the initial ticket status.
Call repository to save the ticket.
```

3. Repository Layer

The repository layer handles data storage.

For this assignment, we will use an in-memory dictionary.

Later, we will replace this with DynamoDB.

4. Model Layer

The model layer defines request and response structures.

For example:

```
CreateTicketRequest  
TicketResponse  
TicketStatus
```

Part 5: Implement Ticket Models

Step 6: Edit app/models/ticket_models.py

Add this code:

```
from enum import Enum  
from typing import Optional  
from pydantic import BaseModel, Field  
  
class TicketStatus(str, Enum):  
    CREATED = "CREATED"  
    WORKFLOW_SELECTED = "WORKFLOW_SELECTED"  
    WORKFLOW_CREATED = "WORKFLOW_CREATED"  
    WORKFLOW_QUEUED = "WORKFLOW_QUEUED"  
    WORKFLOW_RUNNING = "WORKFLOW_RUNNING"  
    RESOLVED = "RESOLVED"  
    FAILED = "FAILED"  
  
class CreateTicketRequest(BaseModel):  
    user_id: str = Field(..., min_length=1)  
    message: str = Field(..., min_length=5)  
    source: Optional[str] = "unknown"  
  
class TicketResponse(BaseModel):  
    ticket_id: str  
    user_id: str  
    message: str  
    source: str  
    status: TicketStatus
```

Explanation

`TicketStatus` defines all allowed ticket statuses.

`CreateTicketRequest` defines the request body for creating a ticket.

`TicketResponse` defines the response returned by the API.

Example request:

```
{
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app"
}
```

Example response:

```
{
  "ticket_id": "ticket_ab12cd34",
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app",
  "status": "CREATED"
}
```

Part 6: Implement In-Memory Ticket Repository

Step 7: Edit `app/repositories/ticket_repository.py`

Add this code:

```
from typing import Dict, List, Optional
from app.models.ticket_models import TicketResponse

class TicketRepository:
    def __init__(self):
        self.tickets: Dict[str, TicketResponse] = {}

    def save(self, ticket: TicketResponse) -> TicketResponse:
        self.tickets[ticket.ticket_id] = ticket
        return ticket

    def get_by_id(self, ticket_id: str) -> Optional[TicketResponse]:
```

```
        return self.tickets.get(ticket_id)

    def list_all(self) -> List[TicketResponse]:
        return list(self.tickets.values())

ticket_repository = TicketRepository()
```

Explanation

For now, we use:

```
self.tickets = {}
```

This is an in-memory dictionary.

The key is:

```
ticket_id
```

The value is the ticket object.

Later, this repository will be replaced by DynamoDB.

Part 7: Implement Ticket Service

Step 8: Edit `app/services/ticket_service.py`

Add this code:

```
from typing import List, Optional
from uuid import uuid4

from app.models.ticket_models import (
    CreateTicketRequest,
    TicketResponse,
    TicketStatus,
)
from app.repositories.ticket_repository import ticket_repository
```

```

class TicketService:
    def create_ticket(self, request: CreateTicketRequest) -> TicketResponse:
        ticket_id = f"ticket_{uuid4().hex[:8]}"

        ticket = TicketResponse(
            ticket_id=ticket_id,
            user_id=request.user_id,
            message=request.message,
            source=request.source or "unknown",
            status=TicketStatus.CREATED,
        )

        return ticket_repository.save(ticket)

    def get_ticket(self, ticket_id: str) -> Optional[TicketResponse]:
        return ticket_repository.get_by_id(ticket_id)

    def list_tickets(self) -> List[TicketResponse]:
        return ticket_repository.list_all()

ticket_service = TicketService()

```

Explanation

The service layer handles business logic.

In this assignment, the business logic is simple:

1. Generate a ticket ID.
2. Create a ticket object.
3. Set the initial status to `CREATED`.
4. Save the ticket through the repository.

Part 8: Implement Ticket Router

Step 9: Edit `app/routers/ticket_router.py`

Add this code:

```

from typing import List

from fastapi import APIRouter, HTTPException, status

```

```

from app.models.ticket_models import CreateTicketRequest, TicketResponse
from app.services.ticket_service import ticket_service

router = APIRouter(
    prefix="/tickets",
    tags=["Tickets"],
)

@router.post(
    "",
    response_model=TicketResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_ticket(request: CreateTicketRequest):
    return ticket_service.create_ticket(request)

@router.get(
    "",
    response_model=List[TicketResponse],
)
def list_tickets():
    return ticket_service.list_tickets()

@router.get(
   ("/{ticket_id}",
    response_model=TicketResponse,
)
def get_ticket(ticket_id: str):
    ticket = ticket_service.get_ticket(ticket_id)

    if ticket is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Ticket not found",
        )

    return ticket

```

Explanation

This router defines three APIs:

```
POST /tickets
```

Creates a ticket.

```
GET /tickets
```

Lists all tickets.

```
GET /tickets/{ticket_id}
```

Gets one ticket by ID.

Part 9: Implement FastAPI Main App

Step 10: Edit app/main.py

Add this code:

```
from fastapi import FastAPI

from app.routers.ticket_router import router as ticket_router

app = FastAPI(
    title="AI Ticket Workflow Platform",
    description="A backend platform for client issue triage and workflow automation.",
    version="0.1.0",
)

app.include_router(ticket_router)

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }
```

Explanation

`FastAPI()` creates the application.

`include_router(ticket_router)` registers all ticket APIs.

`GET /health` is a simple health check API.

Part 10: Run the Server

Step 11: Start FastAPI server

Run:

```
uvicorn app.main:app --reload
```

Expected output should include:

```
Uvicorn running on http://127.0.0.1:8000
```

Open this URL in your browser:

```
http://127.0.0.1:8000/docs
```

This opens FastAPI Swagger UI.

Part 11: Test APIs

Test 1: Health Check

Call:

```
GET /health
```

Expected response:

```
{
  "status": "ok"
}
```

Test 2: Create Ticket

Call:

```
POST /tickets
```

Request body:

```
{
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app"
}
```

Expected response:

```
{
  "ticket_id": "ticket_ab12cd34",
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app",
  "status": "CREATED"
}
```

The actual `ticket_id` will be different because it is generated automatically.

Test 3: Get Ticket By ID

Use the `ticket_id` returned from Test 2.

Call:

```
GET /tickets/{ticket_id}
```

Example:

```
GET /tickets/ticket_ab12cd34
```

Expected response:

```
{
  "ticket_id": "ticket_ab12cd34",
  "user_id": "user_123",
  "message": "Tesla price has not updated for 30 minutes",
  "source": "mobile_app",
  "status": "CREATED"
}
```

Test 4: List All Tickets

Call:

```
GET /tickets
```

Expected response:

```
[
  {
    "ticket_id": "ticket_ab12cd34",
    "user_id": "user_123",
    "message": "Tesla price has not updated for 30 minutes",
    "source": "mobile_app",
    "status": "CREATED"
  }
]
```

Test 5: Get Non-Existing Ticket

Call:

```
GET /tickets/ticket_not_exist
```

Expected status code:

```
404 Not Found
```

Expected response:

```
{
  "detail": "Ticket not found"
}
```

Test 6: Request Validation

Call:

```
POST /tickets
```

Invalid request body:

```
{
  "user_id": "user_123",
  "message": "bad"
}
```

Because `message` must have at least 5 characters, FastAPI should return a validation error.

Expected status code:

```
422 Unprocessable Entity
```

Part 12: Git Commit and Push

Step 12: Check git status

```
git status
```

Step 13: Add files

```
git add .
```

Step 14: Commit

```
git commit -m "Set up FastAPI project and basic ticket APIs"
```

Step 15: Push

```
git push origin main
```

If your branch is named `master`, run:

```
git push origin master
```

Part 13: README Update

Update `README.md` with this content:

```
# AI Ticket Workflow Platform

This project is a backend platform for client issue triage and workflow
automation.

## Current Features

- Health check API
- Create ticket API
- Get ticket by ID API
- List tickets API
- In-memory ticket repository
- Router / service / repository / model project structure

## Tech Stack

- Python
- FastAPI
```

- Uvicorn
- Pydantic

How to Run

```bash

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Open Swagger UI:

<http://127.0.0.1:8000/docs>

## APIs

### Health Check

GET /health

### Create Ticket

POST /tickets

### Get Ticket

GET /tickets/{ticket\_id}

### List Tickets

GET /tickets

## Future Work

- Store tickets in DynamoDB
- Add workflow definitions
- Create workflow runs
- Send workflow run IDs to SQS
- Add worker service
- Add LLM-based issue classification

```

Expected Final Project Structure

Your final project should look like this:

```text
ai-ticket-workflow-platform/
  app/
    __init__.py
    main.py

    models/
      __init__.py
      ticket_models.py

    routers/
      __init__.py
      ticket_router.py

    services/
      __init__.py
      ticket_service.py

    repositories/
      __init__.py
      ticket_repository.py

requirements.txt
README.md
```

Acceptance Criteria

Your assignment is complete if:

1. The project is pushed to GitHub.
2. The FastAPI server starts successfully.
3. `GET /health` returns `{ "status": "ok" }`.
4. `POST /tickets` creates a ticket.
5. `GET /tickets/{ticket_id}` returns the correct ticket.
6. `GET /tickets` returns all tickets.
7. A non-existing ticket returns `404 Not Found`.
8. Invalid request body returns validation error.
9. Code is separated into model, router, service, and repository layers.

10. README includes setup instructions and API list.

Questions to Answer

Please answer these questions after finishing the assignment:

1. What is a backend framework?
 2. Why are we using FastAPI?
 3. What is the responsibility of the router layer?
 4. What is the responsibility of the service layer?
 5. What is the responsibility of the repository layer?
 6. Why are we using in-memory storage for now?
 7. What problem will DynamoDB solve later?
 8. What does `POST /tickets` do?
 9. What does `GET /tickets/{ticket_id}` do?
 10. Why should we not put all code inside `main.py`?
-

Optional Challenge

Add a new API:

```
PATCH /tickets/{ticket_id}/status
```

Request body:

```
{
  "status": "RESOLVED"
}
```

Expected behavior:

1. Find the ticket by `ticket_id`.
2. If the ticket does not exist, return `404 Not Found`.
3. If the ticket exists, update the status.
4. Return the updated ticket.

This challenge will help you practice partial updates and enum validation.